Ipopt and Rust: An exploration of the ecosystem

Seminar Applied Optimization - University of Bern

Student: Horacio Lisdero Scaffino

Supervisor: Ningfeng Zhou

May 21, 2026

Agenda

- A brief tour of Ipopt
  - History
  - Techniques applied
- Why Rust?
- Survey of the Rust ecosystem for optimization
  - ipopt-rs
  - argmin
- A tour of ripopt
  - Context
  - Benchmarks
- Conclusion

Agenda

- A brief tour of Ipopt
  - History
  - Techniques applied
- Why Rust?
- Survey of the Rust ecosystem for optimization
  - ipopt-rs
  - argmin
- A tour of ripopt
  - Context
  - Benchmarks
- Conclusion

Ipopt: Interior Point OPTimizer

Popular open-source non-linear optimization package

Bindings available for:

- AMPL (algebraic modeling language)
- Fortran
- GAMS (general algebraic modeling system)
- Julia
- Matlab / Scilab
- .NET languages: C#, F# and Visual Basic.NET
- Python (several wrappers available)
- R
- Rust

Ipopt: The origins

- PhD thesis by Andreas Wächter in 2002 at Carnegie Mellon[1]
- Since 2002 part of the COIN-OP foundation
- Original implementation in Fortran 77 (1.x and 2.x), now deprecated
- Open source since the start
- Benchmarked against KNITRO (1999)[2] and LOQO (1999)[3]

[1] Wachter 2002.
[2] Byrd, Hribar, and Nocedal 1999.
[3] Vanderbei 1999.

Ipopt: Subsequent paper

- Paper by Wächter and Biegler in 2006[4]
- More focused on the central algorithm of Ipopt
- Features described more in detail
  - Feasibility restoration phase,
  - Inertia correction,
  - Various heuristics...
- More benchmarks: CUTEr test set[5]

[4] Wächter and Biegler 2006.

[5] Gould, Orban, and Toint 2003.

Ipopt: C++ version

- IBM Research decided to invest in an open source re-write of Ipopt in C++
- With the help of Carl Laird, the code was re-implemented from scratch[6]
- 3.x published in Aug 2005
- It is still actively developed on GitHub

[6] History of Ipopt

Ipopt: Summary of applied techniques

- Primal-dual interior-point method with filter line-search globalization
- Sparse symmetric KKT solves with inertia monitoring/correction
- Inertia correction and regularization to obtain the proper search direction
- Feasibility restoration phase for difficult or infeasible iterates
- Second-order correction steps to reduce Maratos-type effects
- Practical heuristics for robustness and faster performance

We will quickly go over the main points for time reasons

Interior Point Methods: Intro

$$\min_{x \in \mathbb{R}^n} \quad \varphi_\mu(x) := f(x) - \mu \sum_{i=1}^{n} \ln\left(x^{(i)}\right) \tag{1}$$

$$\text{s.t.} \quad c(x) = 0 \tag{2}$$

$$x \geq 0 \tag{3}$$

Interior Point Methods: Intro

$$\min_{x \in \mathbb{R}^n} \quad \varphi_\mu(x) := f(x) - \mu \sum_{i=1}^{n} \ln\left(x^{(i)}\right) \tag{1}$$

$$\text{s.t.} \quad c(x) = 0 \tag{2}$$

$$x \geq 0 \tag{3}$$

We recover the Karush-Kuhn-Tucker (KKT) conditions for $\mu = 0$:

$$\nabla f(x) + \nabla c(x)\lambda - z = 0 \tag{4}$$

$$c(x) = 0 \tag{5}$$

$$XZe - \mu e = 0 \tag{6}$$

$$x, z \geq 0, \qquad \lambda \in \mathbb{R}^m, \qquad z \in \mathbb{R}^n,$$

$$Z := \text{diag}(z), e = (1, \ldots, 1)^T$$

n: number of primal variables, m: number of equality constraints

KKT system: Derivation (1/2)

$$F_\mu(x, \lambda, z) = \begin{bmatrix} \nabla f(x) + \nabla c(x)\lambda - z \\ c(x) \\ XZe - \mu e \end{bmatrix} = 0$$

KKT system: Derivation (1/2)

$$F_\mu(x, \lambda, z) = \begin{bmatrix} \nabla f(x) + \nabla c(x)\lambda - z \\ c(x) \\ XZe - \mu e \end{bmatrix} = 0$$

Iteratively solve using Netwon's method:

- Fix a starting point (x_k, λ_k, z_k),
- Calculate the deltas for each dimension $(d_k^x, d_k^\lambda, d_k^z)$
- Use the deltas to move to $(x_{k+1}, \lambda_{k+1}, z_{k+1})$ (more later)

$$F_{\mu_j}(x_k, \lambda_k, z_k) + \nabla F_{\mu_j}(x_k, \lambda_k, z_k)(d_k^x, d_k^\lambda, d_k^z)^T = 0$$

KKT system: Derivation (1/2)

$$F_\mu(x, \lambda, z) = \begin{bmatrix} \nabla f(x) + \nabla c(x)\lambda - z \\ c(x) \\ XZe - \mu e \end{bmatrix} = 0$$

Iteratively solve using Netwon's method:

- Fix a starting point (x_k, λ_k, z_k),
- Calculate the deltas for each dimension $(d_k^x, d_k^\lambda, d_k^z)$
- Use the deltas to move to $(x_{k+1}, \lambda_{k+1}, z_{k+1})$ (more later)

$$F_{\mu_j}(x_k, \lambda_k, z_k) + \nabla F_{\mu_j}(x_k, \lambda_k, z_k)(d_k^x, d_k^\lambda, d_k^z)^T = 0$$

The term $\nabla F_\mu(x, \lambda, z)$ is a 3x3 matrix.
It is the Jacobian of the vector field.
We can move $F_\mu(x, \lambda, z)$ to the other side to get the form $Ax = b$.

KKT system: Derivation (2/2)

$$F_\mu(x,\lambda,z) = \begin{bmatrix} \nabla f(x) + \nabla c(x)\lambda - z \\ c(x) \\ XZe - \mu e \end{bmatrix} = 0$$

$$F_{\mu_j}(x_k,\lambda_k,z_k) + \nabla F_{\mu_j}(x_k,\lambda_k,z_k)(d_k^x, d_k^\lambda, d_k^z)^T = 0$$

$$\begin{bmatrix} W_k & A_k & -I \\ A_k^T & 0 & 0 \\ Z_k & 0 & X_k \end{bmatrix} \begin{bmatrix} d_k^x \\ d_k^\lambda \\ d_k^z \end{bmatrix} = - \begin{bmatrix} \nabla f(x_k) + A_k\lambda_k - z_k \\ c(x_k) \\ X_k Z_k e - \mu_j e \end{bmatrix}$$

$$\begin{aligned} \text{where} \qquad A_k &:= \nabla c(x_k), \\ W_k &:= \nabla^2_{xx}\mathcal{L}(x_k,\lambda_k,z_k), \\ \mathcal{L}(x,\lambda,z) &:= f(x) + c(x)^T\lambda - z. \end{aligned}$$

KKT system: Matrix form

$$\begin{bmatrix} W_k & A_k & -I \\ A_k^T & 0 & 0 \\ Z_k & 0 & X_k \end{bmatrix} \begin{bmatrix} d_k^x \\ d_k^\lambda \\ d_k^z \end{bmatrix} = - \begin{bmatrix} \nabla f(x_k) + A_k \lambda_k - z_k \\ c(x_k) \\ X_k Z_k e - \mu_j e \end{bmatrix}$$

$$\begin{aligned} \text{where} \qquad A_k &:= \nabla c(x_k), \\ W_k &:= \nabla^2_{xx} \mathcal{L}(x_k, \lambda_k, z_k), \\ \mathcal{L}(x, \lambda, z) &:= f(x) + c(x)^T \lambda - z. \end{aligned}$$

But this system can be further simplified!
Multiply out the third equation:

$$\begin{aligned} Z_k d_k^x + X_k d_k^z &= \mu_j e - X_k Z_k e \\ d_k^z = \mu_j X_k^{-1} e - z_k - X_k^{-1} Z_k d_k^x \\ \Sigma_k &:= X_k^{-1} Z_k \end{aligned}$$

KKT system: Symmetric matrix form

Now d_k^z only appears in the first equation as $-Id_k^z = -d_k^z$ which we move to the other side:

$$\begin{aligned}
W_k d_k^x + A_k d_k^\lambda - I d_k^z &= -(\nabla f(x_k) + A_k \lambda_k - z_k) \\
W_k d_k^x + A_k d_k^\lambda &= -(\nabla f(x_k) + A_k \lambda_k - z_k) + d_k^z \\
W_k d_k^x + A_k d_k^\lambda &= -(\nabla f(x_k) + A_k \lambda_k - z_k) + \\
&\quad + \mu_j X_k^{-1} e - z_k - \Sigma_k d_k^x \\
(W_k + \Sigma_k) d_k^x + A_k d_k^\lambda &= -(\nabla f(x_k) + A_k \lambda_k - \mu_j X_k^{-1} e) \\
(W_k + \Sigma_k) d_k^x + A_k d_k^\lambda &= -(\nabla \varphi_{\mu_j}(x_k) + A_k \lambda_k)
\end{aligned}$$

Remember: $\varphi_\mu(x) := f(x) - \mu \sum_{i=\mathbf{1}}^n \ln(x^{(i)})$

Reconstructing the system, we get:

$$\begin{bmatrix} W_k + \Sigma_k & A_k \\ A_k^T & 0 \end{bmatrix} \begin{bmatrix} d_k^x \\ d_k^\lambda \end{bmatrix} = - \begin{bmatrix} \nabla \varphi_{\mu_j}(x_k) + A_k \lambda_k \\ c(x_k) \end{bmatrix} \tag{7}$$

KKT system: Important considerations

$$\begin{bmatrix} W_k + \Sigma_k & A_k \\ A_k^T & 0 \end{bmatrix} \begin{bmatrix} d_k^x \\ d_k^\lambda \end{bmatrix} = - \begin{bmatrix} \nabla \varphi_{\mu_j}(x_k) + A_k \lambda_k \\ c(x_k) \end{bmatrix}$$

- Symmetric system
- Matrix in top-left block projected onto the null space of the constraint Jacobian A_k^T is positive definite
- If A_k does not have full rank, the iteration matrix is singular and a solution may not exist

Solve instead the following system with **inertia correction** for some $\delta_w, \delta_c \geq 0$:

$$\begin{bmatrix} W_k + \Sigma_k + \delta_w I & A_k \\ A_k^T & -\delta_c I \end{bmatrix} \begin{bmatrix} d_k^x \\ d_k^\lambda \end{bmatrix} = - \begin{bmatrix} \nabla \varphi_{\mu_j}(x_k) + A_k \lambda_k \\ c(x_k) \end{bmatrix}$$

Agenda

- A brief tour of Ipopt
  - History
  - Techniques applied
- Why Rust?
- Survey of the Rust ecosystem for optimization
  - ipopt-rs
  - argmin
- A tour of ripopt
  - Context
  - Benchmarks
- Conclusion

What is Rust?

Rust is a multi-paradigm, general-purpose programming language that aims to provide developers with a safe and efficient way to write low-level code.

What is Rust?

Rust is a multi-paradigm, general-purpose programming language that aims to provide developers with a safe and efficient way to write low-level code.

- Memory-safe
- Compiled to machine code, no runtime needed
- High-level simplicity
- Low-level performance (on the same level as C or C++)

Brief timeline of Rust

2007 Started as a side project by Graydon Hoare, a programmer at Mozilla

2009 Mozilla officially started sponsoring the project

2015 First stable version 1.0

2016 Mozilla releases Servo, a browser engine built with Rust

2019 `async`/`await` support stabilized

2021 The Rust Foundation is founded by AWS, Huawei, Google, Microsoft, and Mozilla

2021 The Android Open Source Project encourages the use of Rust for the SO components below the ART

2022 The Linux kernel adds experimental support for Rust alongside C

2023 Microsoft starts introducing Rust into parts of Windows kernel

2025 10 years in a row the most loved programming language in the Stack Overflow Developer Survey

2026 Linux 7.0 released with official Rust support

Memory safety

It achieves memory safety without using a garbage collector or reference counting. Instead, it uses the concept of **ownership** and **borrowing**.

It prevents a wide variety of error classes at compile-time:

- Double free
- Use after free
- Dangling pointers
- Data races
- Passing non-thread-safe variables

If a violation of the compiler rules is found, the program will simply not compile.

Where to use Rust?

TL;DR: to replace C / C++

Like in optimization software where:

- Execution speed directly affects usability
- Memory footprint must stay tight and predictable
- Low-level bugs are costly to diagnose and fix

Agenda

A brief tour of Ipopt
- History
- Techniques applied

Why Rust?

Survey of the Rust ecosystem for optimization
- ipopt-rs
- argmin

A tour of ripopt
- Context
- Benchmarks

Conclusion

What are we looking for?

We will look at different levels of integration on a broad spectrum:

1. Rust bindings to Ipopt
2. an optimization package 100% in Rust *without* IPM
3. an Ipopt-derived implementation in Rust

ipopt-rs

`ipopt-rs`[7] offers a safe Rust interface to Ipopt.

You must install Ipopt on your system first. The accompanying repo contains 3 executable examples.

While it is technically possible to call C/C++ foreign functions from Rust, it is advisable to avoid it to:

- reduce boilerplate and error checking
- keep the `unsafe` code outside of your own code

[7] https://github.com/elrnv/ipopt-rs

An attempt at a simple IPM algorithm

I searched for mature Rust packages that implement an interior point method...

An attempt at a simple IPM algorithm

I searched for mature Rust packages that implement an interior point method...

I could not find any, not even for constrained optimization with other algorithms :(

An attempt at a simple IPM algorithm

I searched for mature Rust packages that implement an interior point method...

I could not find any, not even for constrained optimization with other algorithms :(

But I wanted to see if optimization in pure Rust is possible

memory safety, performance, modern language for low-level applications, ...

An attempt at a simple IPM algorithm

I searched for mature Rust packages that implement an interior point method...

I could not find any, not even for constrained optimization with other algorithms :(

But I wanted to see if optimization in pure Rust is possible

memory safety, performance, modern language for low-level applications, ...

And I wanted to contribute something back to the community,

i.e, **not** throw away the Python code 1 week after the seminar

The original plan

1. Select the most popular optimization package
2. Implement a simple interior-point method
3. Compare it with Ipopt
4. Profit!

argmin

I went on `crates.io`, the Rust package registry, and looked for:

- Optimization packages
- Specific algorithm names
- Linear algebra libraries

The most popular optimization package by a wide margin:

discarded several libraries: dealt only with LP (`good_lp`), used novel algorithms (`clarabel`), or were too small (`interiors`)

argmin: Features

Pure Rust 🦀

We aim at providing a wide range of solvers implemented in pure Rust.

Consistent interface 🔌

Easily plug your optimization problem in different solvers thanks to a consistent interface.

Type agnostic 🔧

Represent your variables, gradients, Jacobians and Hessians using your own types.

Various math backends 🧮

Use Vecs, ndarray, or nalgebra for behind-the-scenes math. Or implement your own backend.

Observers 👀

Observe the progress of your optimization by logging to screen or file or implement your own observer.

Checkpointing 🚩

Mitigate the negative effects of crashes in unstable computing environments by saving regular checkpoints.

Extensible 🪐

Most features can be exchanged for user supplied implementations thanks to Rusts powerful generics and traits.

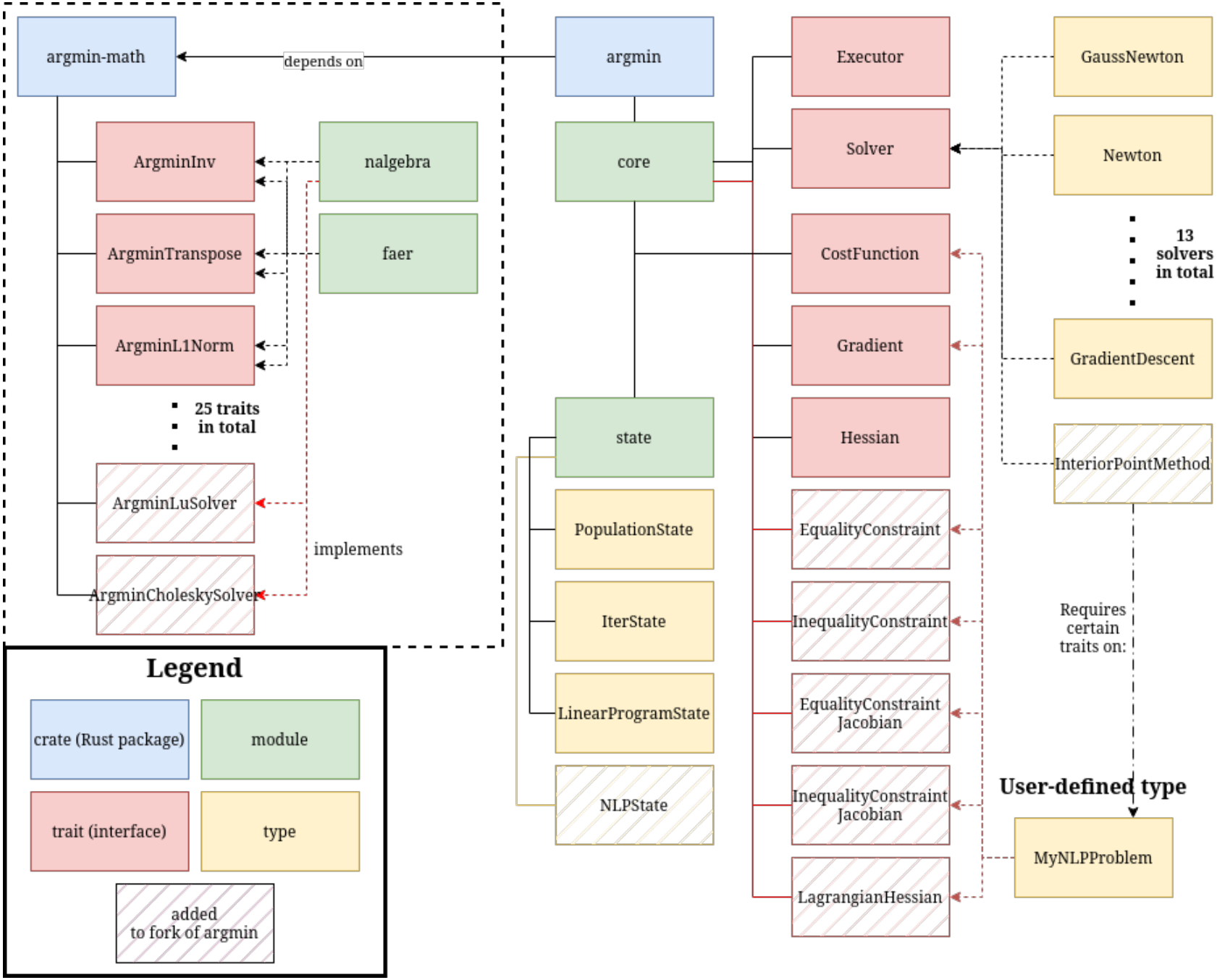
Development framework 🏗️

Use argmins tools and ecosystem to implement your own solver – with all the features mentioned above.

8

[8] Screenshot from the website https://argmin-rs.org/

argmin: Architecture



argmin: Math backends

The math backend is implemented in a separate crate `argmin-math`

It is trait-based: interfaces define the methods required from the linear algebra backend

- `nalgebra`[9]
- `faer`[10]

Following the existing architecture, every matrix operation should be generic over the backend.

But for this toy implementation it would be overkill ...

I decided to take a shortcut and implement a solver tightly coupled to `nalgebra`.

[9] https://nalgebra.rs/

[10] https://faer.veganb.tw/

argmin: Code example

```rust
impl<O, P, G, H, F> Solver<O, IterState<P, G, (), H, (), F>> for Newton<F>
where
    O: Gradient<Param = P, Gradient = G> + Hessian<Param = P, Hessian = H>,
    P: Clone + ArgminScaledSub<P, F, P>,
    H: ArgminInv<H> + ArgminDot<G, P>,
    F: ArgminFloat,
{
    fn next_iter(
        &mut self,
        problem: &mut Problem<O>,
        mut state: IterState<P, G, (), H, (), F>,
    ) -> Result<(IterState<P, G, (), H, (), F>, Option<KV>), Error> {
        let param = state.take_param().ok_or_else(argmin_error_closure!(
            NotInitialized,
            concat!(
                "`Newton` requires an initial parameter vector. ",
                "Please provide an initial guess via `Executor`s `configure` method."
            )
        ))?;
        let grad = problem.gradient(&param)?;
        let hessian = problem.hessian(&param)?;
        let new_param = param.scaled_sub(&self.gamma, &hessian.inv()?.dot(&grad));
        Ok((state.param(new_param), None))
    }
}
```

Notice: The matrix is being inverted at every step (inefficient!). No linear solver $Ax = b$ in the codebase.

Example extension: Add a linear solver

Add a trait:

```rust
pub trait ArgminLuSolve<T> {
    /// Solve linear system using LU decomposition.
    fn lu_solve(&self, rhs: &T) -> Result<T, Error>;
}
```

And the corresponding implementation:

```rust
impl<N, D, C, S> ArgminLuSolve<OMatrix<N, D, C>> for SquareMatrix<N, D, S>
where
    N: ComplexField,
    D: Dim + DimMin<D, Output = D>,
    C: Dim,
    S: Storage<N, D, D>,
    DefaultAllocator: Allocator<N, D, D> + Allocator<N, D, C> + Allocator<N, D>,
{
    #[inline]
    fn lu_solve(&self, rhs: &OMatrix<N, D, C>) -> Result<OMatrix<N, D, C>, Error> {
        let lu = LU::new(self.clone_owned());
        lu.solve(rhs).ok_or_else(|| LuSolveError {}.into())
    }
}
```

This is just scratching the surface of the complexity.
There are lots of new things to keep track of!
And the new `NonLinearProgramState` is generic over 11 types!

argmin: Results

File structure:

- `nalgebra_backend.rs`: All the linear-algebra-specific logic for nalgebra, the only supported backend
- `linesearch.rs`: The custom backtracking linear search. The existing implementation could not be reused
- `iteration.rs`: A single iteration of the interior-point method
- `interiorpointmethod.rs`: The solver. The high-level management of the IPM algorithm.

The forked repo on GitHub follows this structure[11]

The implementation is still incomplete after ∼30 hours of work... All the parts are present but I need to debug it and test it thoroughly. Maybe a summer project?

[11] https://github.com/hlisdero/argmin

Advice on argmin

1. It focuses on *unconstrained* optimization
2. Only the most basic algorithms and it is not optimized
3. Well-structured and very generic (maybe too much?)
4. Math libraries pose a limitation for further development as well

Agenda

A brief tour of Ipopt
- History
- Techniques applied

Why Rust?

Survey of the Rust ecosystem for optimization
- ipopt-rs
- argmin

A tour of ripopt
- Context
- Benchmarks

Conclusion

ripopt: A Rust re-write from February 2026

Coded in ∼60/70 hs during 6 weeks with Claude Code[12]

Size? ∼21,700 lines with no external C/Fortran dependencies

By who? Prof. John Kitchin, Chemical Engineering at Carnegie Mellon[13]

Tool? Opus 4.6 with a 1M context window. Several agents running in parallel. 200 USD per month.

Linear algebra solver?

- First relied on a reimplementation of MUMPS[14]
- Since April 26, separate AI-coded solver by the same author[15]

[12] https://github.com/jkitchin/ripopt
[13] https://engineering.cmu.edu/directory/bios/kitchin-john.html
[14] MUltifrontal Massively Parallel sparse direct Solver: https://mumps-solver.org/
[15] https://github.com/jkitchin/feral

ripopt: AI slop? Why do we care?

Standard benchmarks results claims it "beats" Ipopt

Many new ideas not seen in the original Ipopt paper:

- Mehrotra predictor-corrector with Gondzio centrality corrections[16]
- TRON fast path for bound-constrained problems[17]
- Multi-solver fallback architecture: L-BFGS, Augmented Lagrangian, SQP, and explicit slack reformulation
- Two-phase restoration: fast Gauss-Newton + full NLP restoration subproblem
- Adaptive and monotone barrier parameter strategies with Mehrotra sigma-guided mu updates[18]

[16] Gondzio 1996.
[17] Lin and Moré 1999.
[18] Nocedal, Wächter, and Waltz 2009.

ripopt: Things to have in mind (1/2)

Author's impression:
For small problems there are compelling reasons to be excited about ripopt. For larger problems, there is still a lot of work to do to make it competitive.

Caution!
I have not read all the generated code (I don't even really know how to write Rust, but it is readable). It was generated faster than I could read or understand.

This is pure agentic coding!
Agents touch and run the code. He manages and sets the goals.

License? EPL, same as Ipopt.
Author considers it a derivative work, takes no credit.

ripopt: Things to have in mind (2/2)

It is not a "clean room" development. The AI cites specific parts of the Ipopt C++ codebase.

The AI maintains a ~~manuscript~~ PDF describing the algorithm[19]

The instructions/prompts are public: `CLAUDE.md`

Reflections on development, licensing, concerns, "should you use this?"[20] (not AI!)

A narration of the CHANGELOG[21], maintained by the AI.

---

[19] https://github.com/jkitchin/ripopt/blob/main/manuscript/ripopt.pdf
[20] https://github.com/jkitchin/ripopt/blob/main/reflections.org
[21] https://github.com/jkitchin/ripopt/blob/main/narrative-history.org

ripopt: Benchmarks used

Benchmarks and testing are key to make sure that it actually works!

- Hock-Schittkowski[22] test suite (120 problems / 306 in total)
- CUTEst 745 problems[23]
- Electrical grid optimization problems (4 problems)[24]
- Gas pipeline problems (4 problems)
- Electrolyte thermodynamics problems (13 problems)
- Parameter estimation for a CHO (Chinese Hamster Ovary) cell kinetic model (1 problem)
- Water distribution network problems (6 problems)

We will see how much of this holds up to scrutiny...

[22] Schittkowski 2012; Schittkowski 2008.
[23] Gould, Orban, and Toint 2015.
[24] Zimmerman, Murillo-Sánchez, and Thomas 2010.

How do you even verify these bold claims?

Reviewing the code? Not feasible...

Looking at individual problems?

- Difficult to compare (Visualization of high dimensions? Running iteration by iteration?)
- Codebase changes rapidly
- Everybody comes up with their own "test problems" they care about

Treat the system as a black box. Compare it with Ipopt.
Focus on the big picture and take it from there.

It's just replicating the results, right?

`make benchmark` does not work anymore, targets in Makefile not found #37

Open

hlisdero opened 5 days ago — Last edited by hlisdero

I would like to run the full benchmark suite with the following command to verify the results obtained.

```
make benchmark
```

Unfortunately, none of the benchmark-related targets are present in the Makefile anymore. It also contains some outdated targets like `cutest-install` (script `install_cutest.sh` not present in the repo).

I also tried running a cargo command like:

```
cargo run --bin cutest_suite --features cutest,ipopt-native --release
```

But this lead to the issue of not having CUTEst installed which lead me in turn to the missing bash script `install_cutest.sh`.

How can I work around this?

jkitchin closed this as completed in eb1979e 5 days ago

jkitchin 5 days ago — Last edited by jkitchin — Owner

Thanks for the report. The root `Makefile` *does* still have the `benchmark` / `cutest-*` targets, but they are shims that delegate

Assignees
No one assigne

Labels
No labels

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
No branches or

Participants

How it got fixed

After . . .

- a few hallucinated links,
- understanding how to install CUTEst locally and compiling the Fortran code,
- pointing out missing scripts,
- discovering a Linux-specific issue when linking the Ipopt library,
- testing the automated CUTEst installation flow from scratch,
- finding a collision in the name of two Fortran functions,
- testing every Makefile target,
- spotting the missing Hoch-Schittkowski test suite (removed). . .

CUTEst benchmark summary

Metric	ripopt	Ipopt
Optimal	**562/745** (75.4%)	**574/745** (77.0%)
Acceptable	16	3
Total solved (Optimal + Acceptable)	578 (77.6%)	577 (77.4%)
Solved exclusively	21	20

Both ripopt and Ipopt solved: 557 problems
Matching objectives (<0.01%): 537/557
Acceptable at worse local min: 4

Note: ripopt uses fallback strategies (L-BFGS Hessian, Augmented Lagrangian, SQP, slack reformulation) that Ipopt does not have, which accounts for much of the Acceptable count difference.

Acceptable? This was inherited from Ipopt's own convergence framework: a solve is *Optimal* if it meets tight tolerances, and *Acceptable* if it meets relaxed tolerances.

And in the repo?

My latest run (May 11th):

Metric	ripopt	lpopt
Optimal	**562/745** (75.4%)	**574/745** (77.0%)
Acceptable	16	3
Total solved (Optimal + Acceptable)	578 (77.6%)	577 (77.4%)
Solved exclusively	21	20

The repo (May 18th):

Metric	ripopt	lpopt
Optimal	**568/745** (76.2%)	**572/745** (76.8%)
Acceptable	20	5
Total solved (Optimal + Acceptable)	588 (78.9%)	575 (77.2%)
Solved exclusively	23	27

CUTEst performance

Metric	ripopt	Ipopt
Median time	289 µs	2.9 ms
Total time	29.76 s	27.98 s
Mean iterations	38.6	37.1
Median iterations	13	12

- **Geometric mean speedup**: 7.5×
- **Median speedup**: 10.5×
- ripopt faster: 478/541 (88%)
- ripopt 10×+ faster: 288/541
- Ipopt faster: 63/541

Comparison with committed report: The results in the repo look slightly better. The difference could be due to the different hardware, a regression after new commits, and partly AI hallucination.

Other test suites

Suite	N	ripopt	Ipopt	r-only	i-only	Both	Match
CUTEst	727	561 (77.2%)	561 (77.2%)	20	20	541	522/541
Electrolyte	13	13 (100.0%)	12 (92.3%)	1	0	12	11/12
Grid	4	4 (100.0%)	4 (100.0%)	0	0	4	4/4
CHO	1	0 (0.0%)	0 (0.0%)	0	0	0	0/1

Remarks:

- r-only: number of problems which only ripopt could solve
- i-only: number of problems which only Ipopt could solve
- CHO problem also not solved in report in repo. Probably a bug?

Electrolyte and Grid performance

Electrolyte suite (12 problems)[a]

Metric	ripopt	Ipopt
Median time	121 µs	1.5 ms
Total time	30.1 ms	82.0 ms
Mean iterations	203.1	27.9
Median iterations	7	7

- **Geometric mean speedup**: 11.1×
- **Median speedup**: 14.8×
- ripopt faster: 11/12 (92%)
- ripopt 10×+ faster: 11/12
- Ipopt faster: 1/12

Grid suite (4 problems)

Metric	ripopt	Ipopt
Median time	19.0 ms	16.2 ms
Total time	114.7 ms	157.4 ms
Mean iterations	15.8	12.5
Median iterations	19	14

- **Geometric mean speedup**: 1.8×
- **Median speedup**: 1.8×
- ripopt faster: 3/4 (75%)
- ripopt 10×+ faster: 0/4
- Ipopt faster: 1/4

[a] 12 instead of 13, unclear why

Large scale problems: Solver comparison

Problem	n	m	ripopt	iters	time	Ipopt	iters	time
Rosenbrock 500	500	0	Optimal	751	0.127s	Optimal	749	0.280s
SparseQP 1K	500	500	Optimal	6	0.007s	Optimal	6	0.006s
Bratu 1K	1,000	998	Optimal	2	0.004s	Optimal	2	0.003s
OptControl 2.5K	2,499	1,250	Optimal	1	0.017s	Optimal	1	0.004s
Rosenbrock 5K	5,000	0	MaxIterations	2999	5.340s	Failed	3000	7.093s
Poisson 2.5K	5,000	2,500	Optimal	1	0.104s	Optimal	1	0.018s
Bratu 10K	10,000	9,998	Optimal	1	0.039s	Optimal	2	0.023s
OptControl 20K	19,999	10,000	Optimal	1	0.167s	Optimal	1	0.034s
Poisson 50K	49,928	24,964	Optimal	1	2.416s	Optimal	1	0.248s
SparseQP 100K	50,000	50,000	Optimal	6	1.192s	Optimal	6	0.638s

- ripopt: **9/10 solved** in 9.4s total
- Ipopt: **9/10 solved** in 8.3s total
- Speedups (time iopt/time ripopt):
  2.2x, 0.8x, 0.7x, 0.3x, 1.3x, 0.2x, 0.6x, 0.2x, 0.1x, 0.5x

Bottom line: ripopt is slower with larger problems due to less performant linear solver

How would you troubleshoot an issue?

Ipopt reports more than symply "Optimal" / "Not optimal".
The return value (as we will see shortly) is the starting point.

The repo contains several executables that demonstrates issues with specific problems from the test suites. These act as regression tests as development progresses.

Nonetheless, the number of unit tests is relative small: 565 tests
And test coverage results (*sim*78.78%) may be misleading.

CUTEst Suite: failure modes

Failure Mode	ripopt	Ipopt
ErrorInStepComputation	0	2
EvaluationError	3	0
Infeasible	0	10
InvalidNumberDetected	0	1
IpoptStatus(-10)	0	123
IpoptStatus(4)	0	3
LocalInfeasibility	27	0
MaxIterations	22	12
MaxTimeExceeded	15	0
NumericalError	11	0
RestorationFailed	70	4
StopAtTinyStep	1	0
Timeout	17	11

IpoptStatus(-10) = not enough degrees of freedom (typically overconstrained model). IpoptStatus(4) = diverging iterates. InvalidNumberDetected = NaN/Inf in functions or derivatives. RestorationFailed = feasibility restoration failed.

Advice on ripopt

1. Not ready for production
2. For small problems usable, although codebase unstable
3. For large problems still not competitive
4. If people start using it, it may become better

Agenda

- A brief tour of Ipopt
  - History
  - Techniques applied
- Why Rust?
- Survey of the Rust ecosystem for optimization
  - ipopt-rs
  - argmin
- A tour of ripopt
  - Context
  - Benchmarks
- Conclusion

Conclusion

Ipopt's performance may be improved by leveraging Rust's zero cost abstractions

but you need a mature linear solver in Rust as well

Implementation of a state-of-the-art optimization package may be in the reach of modern AI agentic coding tools

but it does *NOT* replace human know-how and decades of careful optimization

Questions?

Links

Presentation:
https://github.com/hlisdero/JMCS_Seminar_Applied_Optimization

Modified argmin:
https://github.com/hlisdero/argmin

ripopt:
https://github.com/jkitchin/ripopt

ipopt-rs:
https://github.com/elrnv/ipopt-rs

Bibliography I

- Byrd, Richard H, Mary E Hribar, and Jorge Nocedal (1999). "An interior point algorithm for large-scale nonlinear programming". In: *SIAM Journal on Optimization* 9.4, pp. 877–900.
- COIN-OR Foundation (2026). *Ipopt: Interior Point OPTimizer*. URL: https://coin-or.github.io/Ipopt/.
- Crozet, Sébastien (n.d.). *nalgebra: Linear algebra library for Rust*. https://nalgebra.rs/. Accessed: 2026-03-28.
- Foundation, COIN-OR (n.d.). *COIN-OP IPOPT*. https://github.com/coin-or/Ipopt. Accessed: 2026-04-15.
- Gondzio, Jacek (1996). "Multiple centrality corrections in a primal-dual method for linear programming". In: *Computational optimization and applications* 6.2, pp. 137–156.
- Gould, Nicholas IM, Dominique Orban, and Philippe L Toint (2003). "CUTEr and SifDec: A constrained and unconstrained testing environment, revisited". In: *ACM Transactions on Mathematical Software (TOMS)* 29.4, pp. 373–394.

Bibliography II

- Gould, Nicholas IM, Dominique Orban, and Philippe L Toint (2015). "CUTEst: a constrained and unconstrained testing environment with safe threads for mathematical optimization". In: *Computational optimization and applications* 60.3, pp. 545–557.
- Kitchin, John (n.d.[a]). *feral: Factored Error-Resistant Algebra Library*. https://github.com/jkitchin/feral. Accessed: 2026-05-15.
- — (n.d.[b]). *ripopt: A memory-safe interior point optimizer written in Rust, inspired by Ipopt*. https://github.com/jkitchin/ripopt. Accessed: 2026-04-11.
- Kroboth, Stefan (n.d.). *argmin: Mathematical optimization in pure Rust*. https://argmin-rs.org/. Accessed: 2026-03-28.

Bibliography III

- Larionov, Egor (n.d.). *ipopt-rs: A safe Rust interface to the Ipopt non-linear optimization library*. https://github.com/elrnv/ipopt-rs. Accessed: 2026-03-27.
- Lin, Chih-Jen and Jorge J Moré (1999). "Newton's method for large bound-constrained optimization problems". In: *SIAM Journal on Optimization* 9.4, pp. 1100–1127.
- Nocedal, Jorge, Andreas Wächter, and Richard A. Waltz (2009). "Adaptive Barrier Update Strategies for Nonlinear Interior Methods". In: *SIAM Journal on Optimization* 19.4, pp. 1674–1693. DOI: 10.1137/060649513. eprint: https://doi.org/10.1137/060649513. URL: https://doi.org/10.1137/060649513.
- Nocedal, Jorge and Stephen J Wright (2006). *Numerical optimization*. Springer.

Bibliography IV

- Quiñones, Sarah (n.d.). *faer: linear algebra library for rust*. https://faer.veganb.tw/. Accessed: 2026-03-28.
- Schittkowski, Klaus (2008). *306 Test Problems for Nonlinear Programming with Optimal Solutions*. User's Guide, University of Bayreuth, Department of Computer Science, December 10, 2008.
- — (2012). *Nonlinear programming codes: information, tests, performance*. Springer Science & Business Media.
- Technologies, Mumps (n.d.). *MUltifrontal Massively Parallel sparse direct Solver*. https://mumps-solver.org/. Accessed: 2026-05-15.
- Vanderbei, Robert J (1999). "LOQO: An interior point code for quadratic programming". In: *Optimization methods and software* 11.1-4, pp. 451–484.

Bibliography V

- Vigerske, Stefan (n.d.). *Mixed-Integer Nonlinear Programming Library (MINLPLib)*. https://www.minlplib.org/. Accessed: 2026-05-15.
- Wachter, Andreas (2002). *An interior point algorithm for large-scale nonlinear optimization with applications in process engineering*. Carnegie Mellon University.
- Wächter, Andreas and Lorenz T Biegler (2006). "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming". In: *Mathematical programming* 106.1, pp. 25–57.
- Zimmerman, Ray Daniel, Carlos Edmundo Murillo-Sánchez, and Robert John Thomas (2010). "MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education". In: *IEEE Transactions on power systems* 26.1, pp. 12–19.

Appendix

- More slides on what needs to be added to argmin

No constraints?

```rust
/// Defines equality constraints for a problem.
///
/// It follows the form h(x) = (0, ... , 0), where h is a vector field.
pub trait EqualityConstraint {
    /// Type of the parameter vector
    type Param;
    /// Type of the return value of the equality constraints
    type Output;

    /// Compute all equality constraints
    fn equality_constraint(&self, param: &Self::Param) -> Result<Self::Output, Error>;
}
```

And their derivatives ...

```rust
/// Defines the Jacobian of the vector field representing the equality constraints for a
↪ problem.
pub trait EqualityConstraintJacobian {
    /// Type of the parameter vector
    type Param;
    /// Type of the Jacobian
    type Jacobian;

    /// Compute the Jacobian of the equality constraints
    fn equality_constraint_jacobian(&self, param: &Self::Param) -> Result<Self::Jacobian,
    ↪ Error>;

    bulk!(equality_constraint_jacobian, Self::Param, Self::Jacobian);
}
```

And the Hessian of the Lagrangian ...

The existing `Hessian` is not enough. We need Lagrange multipliers as well ...

```rust
pub trait LagrangianHessian {
    /// Type of the parameter vector
    type Param;
    /// Type of the multipliers associated with equality constraints
    type MultipliersEq;
    /// Type of the multipliers associated with inequality constraints
    type MultipliersIneq;
    /// Type of the Hessian
    type Hessian;

    /// Compute the Hessian of the Lagrangian
    fn lagrangian_hessian(
        &self,
        param: &Self::Param,
        multipliers_eq: &Self::MultipliersEq,
        multipliers_ineq: &Self::MultipliersIneq,
    ) -> Result<Self::Hessian, Error>;

    bulk!(
        lagrangian_hessian,
        Self::Param,
        Self::MultipliersEq,
        Self::MultipliersIneq,
        Self::Hessian
    );
}
```

And iteration state for NLP ...

Lots of new things to keep track of!

- `slacks`
- `best_slacks`
- `lagrangian_hessian`
- `equality_constraints`
- `inequality_constraints`
- `equality_constraint_jacobian`
- `inequality_constraint_jacobian`
- `mu`
- `lambda_eq`
- `lambda_ineq`
- `alpha_primal`
- `alpha_dual`
- `inf_pr`
- `inf_du`
- `compl_inf`

The resulting `NonLinearProgramState` is generic over 11 types!

NonLinearProgramState

```rust
pub struct NonLinearProgramState<
    P,
    G,
    LH,
    F,
    EqC = (),
    IneqC = (),
    S = (),
    EqCJ = (),
    IneqCJ = (),
    LambdaEq = (),
    LambdaIneq = (),
> {
  /// Current parameter vector
  pub param: Option<P>,
  /// Previous parameter vector
  pub prev_param: Option<P>,
  /// Current slack variables
  pub slacks: Option<S>,
  /// Previous slack variables
  pub prev_slacks: Option<S>,
  /// Current best parameter vector
  pub best_param: Option<P>,
  /// Previous best parameter vector
  pub prev_best_param: Option<P>,
  /// Current best slack variables
  pub best_slacks: Option<S>,
  /// Previous best slack variables
  pub prev_best_slacks: Option<S>,
  /// Current cost function value
  pub cost: F,
  /// Previous cost function value
  pub prev_cost: F,
  /// Current best cost function value
  pub best_cost: F,
  ....
```